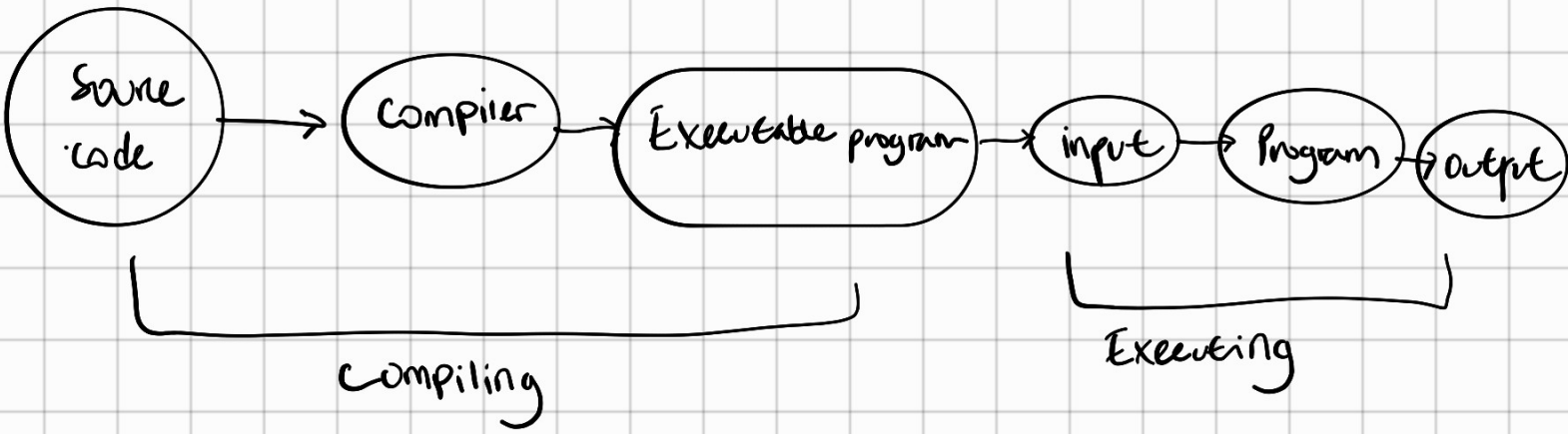
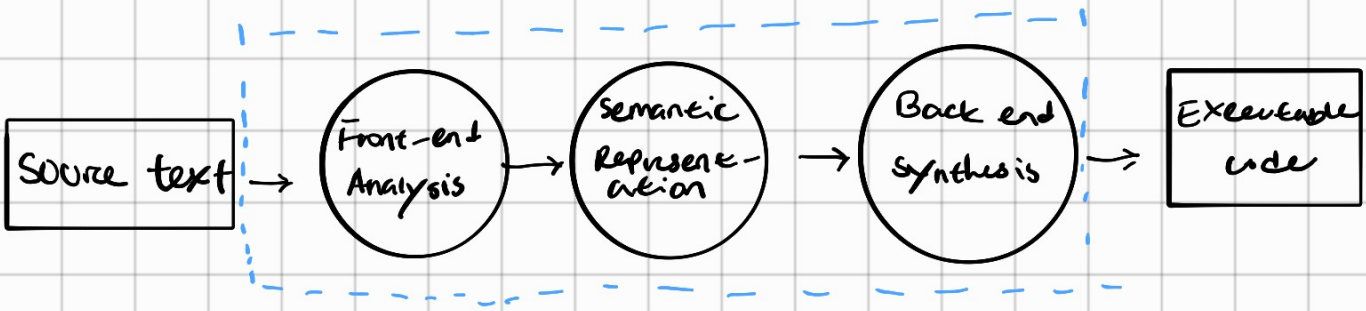
Compilers

↳ Compilers translate source code into machine language.



↳ Structure of a compiler



Lexes / Lexical Analysis

Q1 : lexical analysis & Tokenization

- LETTER = a | b | .... | z
- DIGIT = 0 | 1 | .... | 9
- ID = LETTER (LETTER | DIGIT)*
- INT = DIGIT (DIGIT)*
- Assign = ":="
- Semicolon = ";"
- Plus = "+"
- Mult = "*"
- minus = "_"

eg: "value1 = limit2 + 99 * index ;"

↳ ["value1", "=", "limit2", "+", "99", "*", "index", ";"]

Lexemes	Token
value1	ID
=	assign
limit2	ID
+	plus
99	DIGIT
*	mult
index	ID
;	semicolon

- RegEx & Language.

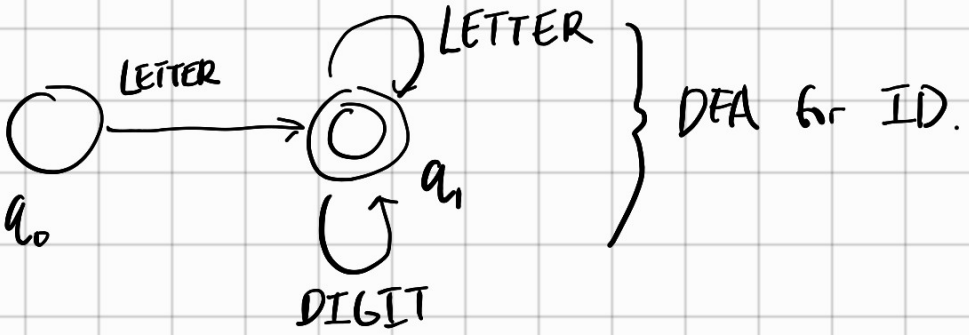
eg: $\sum = \{0, 1\}$, describe the langex $0^* 1 0^*$

the language $0^* 1 0^*$ denotes that it has to start with zero or more "0"s followed by a 1 in the middle and the suffix must also end with zero or more "0"s.

- DFA / NFA.

↳ Create a DFA / NFA for ID token

- ID = LETTER (LETTER | DIGIT)*



eg: A programmer types "limit + * 99"

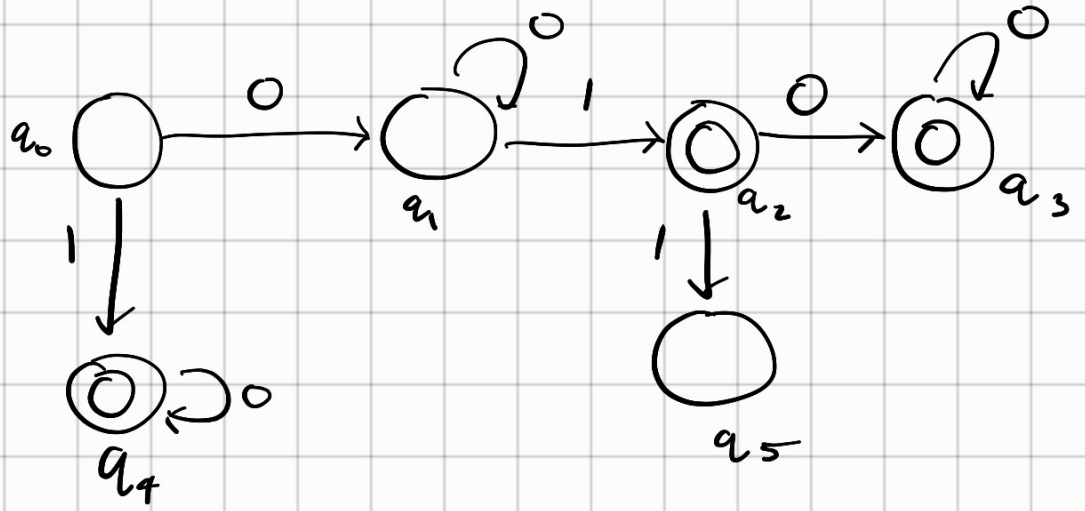
↳ The lexical analyzer will succesfully recognize the statement

↳ However, it is not a valid programming statement

↳ The parser is the one responsible for determining the invalid structure.

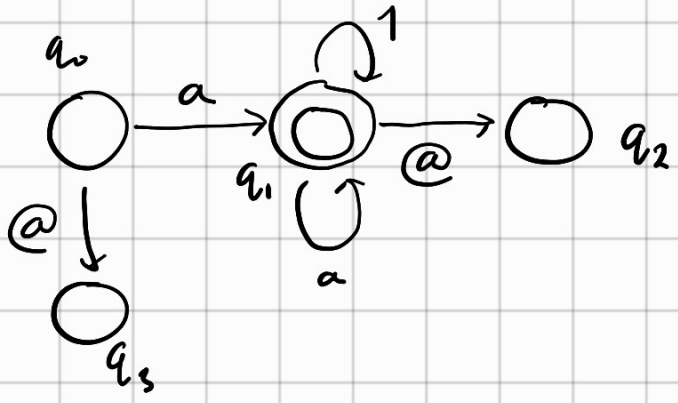
DFAs & NFAs

eg: $\Sigma = \{0, 1\}$, 0^*10^*



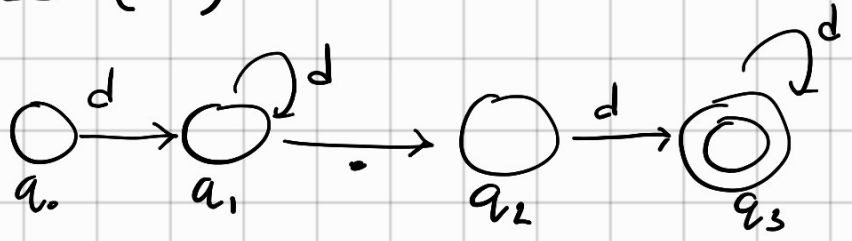
- ID = LETTER (LETTER | DIGIT)*

$\Sigma = \{a, 1, @\}$



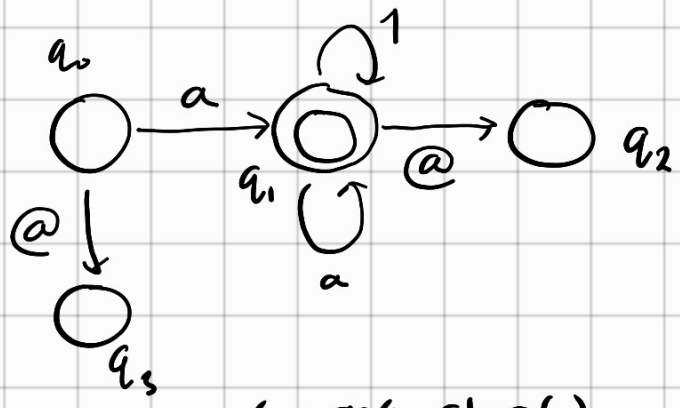
- ID = DIGIT (DIGIT)* . DIGIT(DIGIT)*

$\Sigma = \{d, .\}$



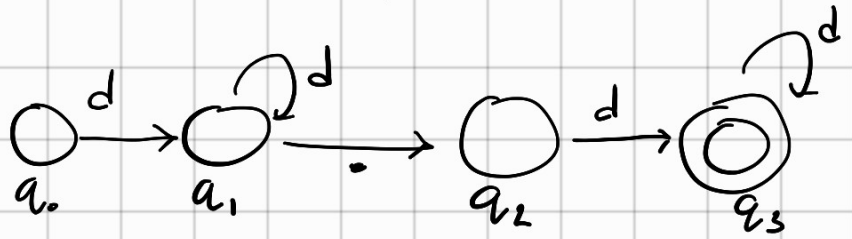
ID = LETTER (LETTER | DIGIT)*

$\Sigma = \{a, 1, @\}$




```
C = get_char()
if ('a' <= C <= 'z') or ('A' <= C <= 'Z'):
    while (True):
        if ('a' <= C <= 'z') or ('A' <= C <= 'Z') or ('0' <= C <= '9'):
            Continue
        else:
            break
Return "ID"
```

eg: REAL = DIGIT (DIGIT)* . DIGIT (DIGIT)*




```
C = get_char()
if ("0" <= C <= "9"):
    while True:
        if ("0" <= C <= "9"):
            continue
        elif (C == "."):
            break
        else:
```

```
        return "Error"

    c=get_char()
    if ('0'<=c<= '9') :
        while True:
            if('0'<=c <= '9'):
                continue
            else :
                Break
            return " REAL"

    else:
        return "Error"
Else :
 return "Error"
```

* Parser

CFG

Q1:

eg: <stmt> → ID ASSIGN <expr> SEMICOLON

<expr> → <term> | <term> PLUS <expr>

<term> → <factor> | <factor> MULT <term>

<factor> → ID | INTEGER

Drive: ID ASSIGN INTEGER PLUS ID SEMICOLON

↳ <stmt> → ID ASSIGN <expr> SEMICOLON

↳ ID ASSIGN <term> PLUS <expr> SEMICOLON

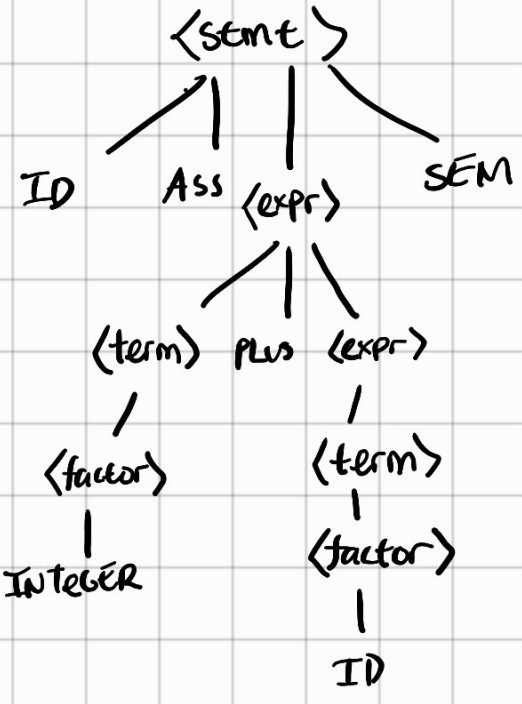
↳ ID ASSIGN <factor> PLUS <term> SEMICOLON

↳ ID ASSIGN INTEGER PLUS <factor> SEMICOLON

↳ ID ASSIGN INTEGER PLUS ID SEMICOLON

} Left-most Derivation.

* Parse Tree for CFG



<stmt> → ID ASSIGN <expr> SEMICOLON

<expr> → <term> | <term> PLUS <expr>

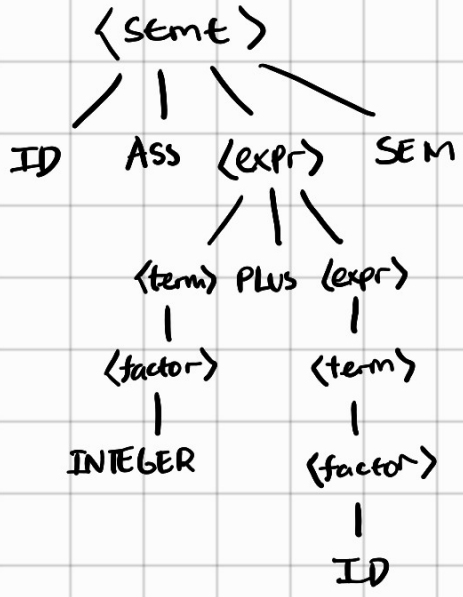
<term> → <factor> | <factor> MULT <term>

<factor> → ID | INTEGER

Given: ID ASSIGN INTEGER PLUS ID SEMICOLON

↳ <stmt> → ID ASSIGN <expr> SEMICOLON
↳ ID ASSIGN <term> PLUS <expr> SEMICOLON
↳ ID ASSIGN <term> PLUS <term> SEMICOLON
↳ ID ASSIGN <term> PLUS <factor> SEMICOLON
↳ ID ASSIGN <term> PLUS ID SEMICOLON
↳ ID ASSIGN <factor> PLUS ID SEMICOLON
↳ ID ASSIGN INTEGER PLUS ID SEMICOLON

} Right-Most Derivation



Accept or Reject

<stmt> → ID ASSIGN <expr> SEMICOLON

<expr> → <term> | <term> PLUS <expr>

<term> → <factor> | <factor> MULT <term>

<factor> → ID | INTEGER

- ID ASSIGN INTEGER MULT PLUS ID SEMICOLON (Reject)
- INTEGER ASSIGN ID SEMICOLON (Reject)
- ID ASSIGN ID SEMICOLON (Accept)
- ID ASSIGN INTEGER PLUS ID MULT INTEGER SEMICOLON (Accept)

eg:

area = (width + 5) * length =>

ID ASSIGN LPAREN ID PLUS INTEGER RPAREN MULT ID

<stmt> → ID ASSIGN <expr>

<expr> → <term> | <term> PLUS <expr>

<term> → <factor> | <factor> MULT <term>

<factor> → ID | INTEGER | LPAREN <expr> RPAREN

<stmt> → ID ASSIGN <expr>

↳ ID ASSIGN <term>

↳ ID ASSIGN <factor> MULT <term>

↳ ID ASSIGN LPAREN <expr> RPAREN MULT <term>

↳ ID ASSIGN LPAREN <term> PLUS <expr> RPAREN MULT <term>

↳ ID ASSIGN LPAREN <factor> PLUS <expr> RPAREN MULT <term>

↳ ID ASSIGN LPAREN ID PLUS <expr> RPAREN MULT <term>

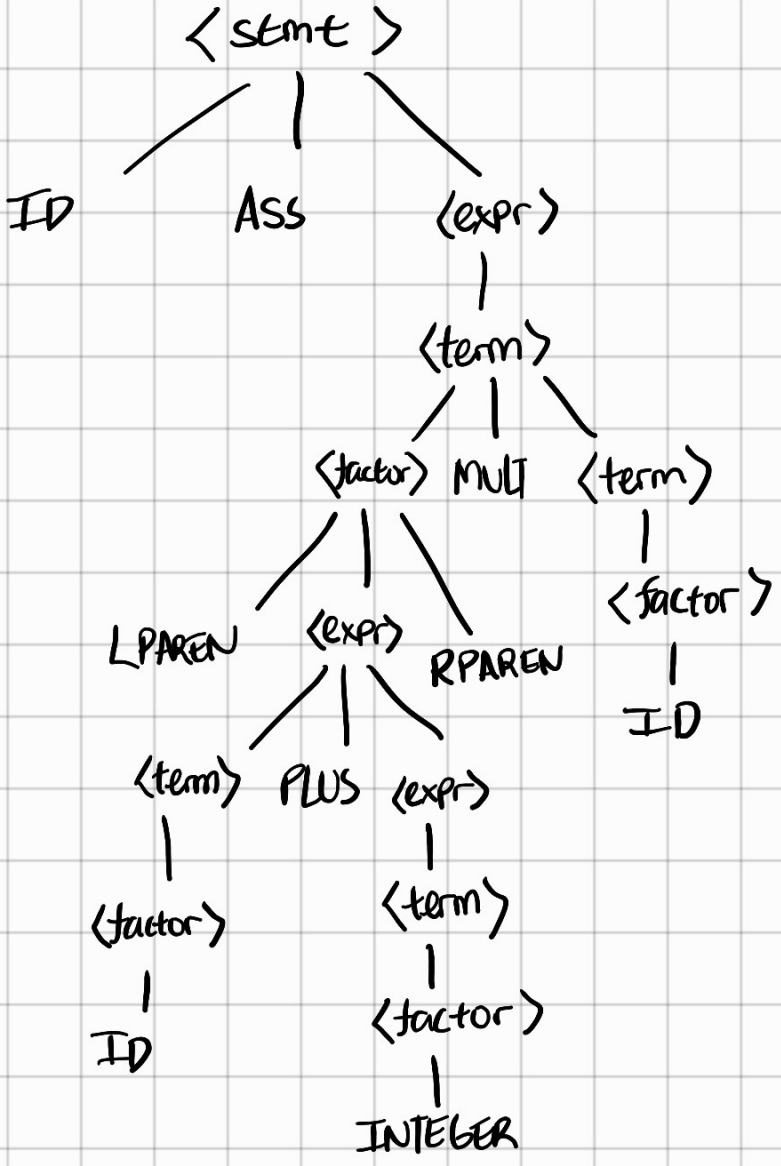
↳ ID ASSIGN LPAREN ID PLUS <term> RPAREN MULT <term>

↳ ID ASSIGN LPAREN ID PLUS <factor> RPAREN MULT <term>

↳ ID ASSIGN LPAREN ID PLUS INTEGER RPAREN MULT ⟨term⟩

↳ ID ASSIGN LPAREN ID PLUS INTEGER RPAREN MULT ⟨factor⟩

↳ ID ASSIGN LPAREN ID PLUS INTEGER RPAREN MULT ID



⟨stmt⟩ → ID ASSIGN ⟨expr⟩ SEMICOLON

⟨expr⟩ → ⟨term⟩ | ⟨term⟩ PLUS ⟨expr⟩

⟨term⟩ → ⟨factor⟩ | ⟨factor⟩ MULT ⟨term⟩

⟨factor⟩ → ID | INTEGER

ID ASSIGN ID PLUS INTEGER MULT ID SEMICOLON

CompTech Quiz 1.

Q1A.

Characters

↓

Lexical Analysis / Lexer

↓

Tokens

↓

Parser / Syntax Analysis

↓

grammatical structure.

1B. Lexical Analysis is the first step in compiler design. It assigns tokens as keywords using RegEx to define the language that is recognized. However, a Lexer only produces the tokens, but not the relationship between them. The Parser during syntax analysis creates a CFG that determines the grammatical structure of the tokens produced during Lexical analysis or by the Lexer.

Q2.

let → LET

result → ID

= → ASSIGN

Price → ID

+ → PLUS

25 → INTEGER

* → MULT

count → ID

; → SEMICOLON

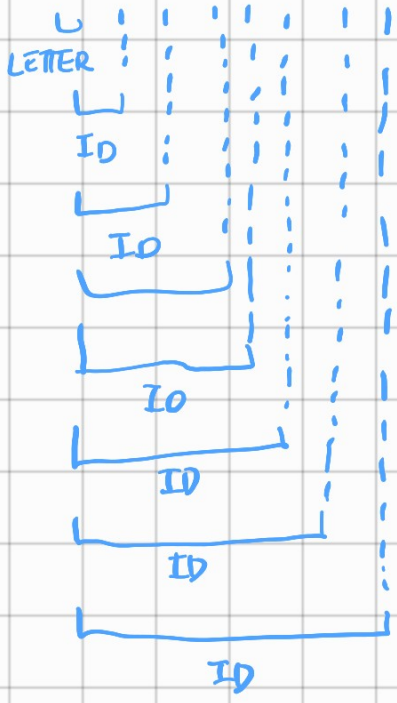
↳ The sequential order is preserved, the categorical meaning as well as the semantic values of ID & INTEGER. The raw character formatting (eg: whitespace, newline, etc.) is abstracted away that separates the lexemes.

Q3.

result 25

↳ yes

↳ result 25



How bout '25result'

NO, '25result' is not valid because ID demands that it strictly has to start with a LETTER, not a DIGIT. The lexer would maybe recognize '25' as INTEGER and 'result' as ID.

Q4A (scanner code)

```
if c.isalpha():
    start = i
    i += 1
    while i < len(src) and (src[i].isalpha() or src[i].isdigit()):

lexeme = src[start:i]
tokens.append(("ID", lexeme))
```

4B.

LETTER directly corresponds to "if c.isalpha()", the condition in which ID has to start with a LETTER or alphabet.

"while i < len(src) and (src[i].isalpha() or src[i].isdigit()):", directly corresponds to the repetition part, (LETTER | DIGIT)*, the program will loop every time it sees another LETTER or DIGIT after the initial starting condition.

4c.

Regular Expression
↓
Regular Language
↓
Recognition Process
↓
Scanner code.

Firstly, the RegEx defines the theoretical rule of the language, defining the characters that will be accepted by the computer. Regular language is the set of strings allowed by the rule, eg: Results. the recognition process checks each character in the string using a Finite-state machine or DFAs. The scanner code is just an executable implementation of the state-machine that can be executed and run on code.

Q5 (Become the Parser)

eg: LET ID ASSIGN ID PLUS INTEGER MULT ID SEMICOLON

↳ ⟨sent⟩ → LET ID ASSIGN ⟨expr⟩ SEMICOLON
↳ LET ID ASSIGN ⟨term⟩ PLUS ⟨expr⟩ SEMICOLON
→ LET ID ASSIGN ⟨factor⟩ PLUS ⟨expr⟩ SEMICOLON
→ LET ID ASSIGN ID PLUS ⟨expr⟩ SEMICOLON
→ LET ID ASSIGN ID PLUS ⟨term⟩ SEMICOLON
→ LET ID ASSIGN ID PLUS ⟨factor⟩ MULT ⟨term⟩ SEMICOLON
→ LET ID ASSIGN ID PLUS INTEGER MULT ⟨term⟩ SEMICOLON
→ LET ID ASSIGN ID PLUS INTEGER MULT ⟨factor⟩ SEMICOLON
→ LET ID ASSIGN ID PLUS INTEGER MULT ID SEMICOLON

ID → <factor>

INTEGER → <factor>

INTEGER MULT ID → <term> => <factor> MULT <term>

ID PLUS INTEGER MULT ID → <expr>

LET ID ASSIGN ID PLUS INTEGER MULT ID SEMICOLON → <stmt>

Q6A.

'Price + 25 * count

ID PLUS INTEGER MULT ID

the '<expr> → <term> PLUS <expr>' allows parsing to continue.

6B.

the rule strictly states that PLUS is followed by <expr>, since <expr> resolve in to <term> & <factor> eventually, it must call the 'Parse_term()' & 'Parse_factor()' function rather than accepting a next token (eg: an operator). (to find a valid INTEGER or ID.

6C.

A CFG/EBNF specification is the theoretical blueprint of how the language should behave and which grammar structures are legal. The recursive-descent parser is the actual executable machine that validates the token streams against the blueprint.

Q7A (Lexical Analysis)

let result = price +* count ; LET ID ASSIGN ID PLUS MULT ID SEMI -

NO, Lexical analysis didn't fail, Lexical analysis only recognizes the language and produce tokens. The lexer will sucessfully recognize the lexemes regardless of their grammatical structure.

7B (Syntax Analysis)

1. The parsing is still valid up uneil the PLUS symbol.
2. The MULT symbol after it reads PLUS
3. PLUS must be followed strictly by ⟨expr⟩ → ⟨term⟩ → ⟨factor⟩ MULT ⟨term⟩

   thus, PLUS & MULT can't be next to each other.

4. The grammar doesn't have a rule where two operators can be adjacent.

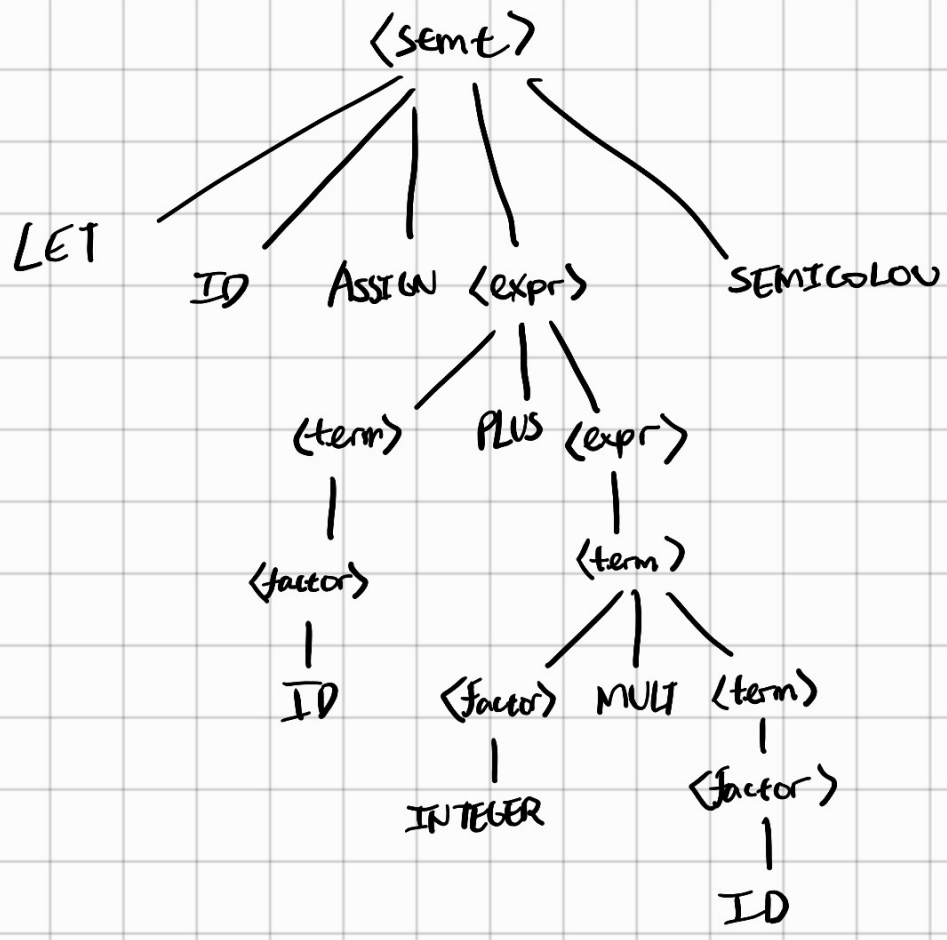
7C. The program fails not because of an unrecognizable token, but because of an illegal relationship between recognized tokens.

Q8.

Even if a lexer can recognize every word and symbol as tokens, the program may not always be syntactically valid. In the statement, 'let result = price + * count', the lexer will successfully recognize the characters and produce a stream of valid tokens. However, during the parsing of these tokens by the parser against a grammar rule / CFG, it ultimately fails the parsing stage on the MULT symbol because there is no rule that states that two operators can be adjacent to each other.

eg: LET ID ASSIGN ID PLUS INTEGER MULT ID SEMICOLON

↳ ⟨semt⟩ → LET ID ASSIGN ⟨expr⟩ SEMICOLON
↳ LET ID ASSIGN ⟨term⟩ PLUS ⟨expr⟩ SEMICOLON
→ LET ID ASSIGN ⟨factor⟩ PLUS ⟨expr⟩ SEMICOLON
→ LET ID ASSIGN ID PLUS ⟨expr⟩ SEMICOLON
→ LET ID ASSIGN ID PLUS ⟨term⟩ SEMICOLON
→ LET ID ASSIGN ID PLUS ⟨factor⟩ MULT ⟨term⟩ SEMICOLON
→ LET ID ASSIGN ID PLUS INTEGER MULT ⟨term⟩ SEMICOLON
→ LET ID ASSIGN ID PLUS INTEGER MULT ⟨factor⟩ SEMICOLON
→ LET ID ASSIGN ID PLUS INTEGER MULT ID SEMICOLON



LET ID ASSIGN ID PLUS INTEGER MULT ID SEMICOLON

⟨seme⟩ → LET ID ASSIGN ⟨expr⟩ SEMICOLON

↳ LET ID ASSIGN ⟨term⟩ PLUS ⟨expr⟩ SEMICOLON

↳ LET ID ASSIGN ⟨term⟩ PLUS ⟨term⟩ SEMICOLON

↳ LET ID ASSIGN ⟨term⟩ PLUS ⟨factor⟩ MULT ⟨term⟩ SEMICOLON

↳ LET ID ASSIGN ⟨term⟩ PLUS ⟨factor⟩ MULT ⟨factor⟩ SEMICOLON

↳ LET ID ASSIGN ⟨term⟩ PLUS ⟨factor⟩ MULT ID SEMICOLON

↳ LET ID ASSIGN ⟨term⟩ PLUS INTEGER MULT ID SEMICOLON

↳ LET ID ASSIGN ⟨factor⟩ PLUS INTEGER MULT ID SEMICOLON

↳ LET ID ASSIGN ID PLUS INTEGER MULT ID SEMICOLON

